

Module 0

Installation and Setup

Learning objectives

- Set up development environment and verify installation

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["System Requirements and Prer"]

T2["Python Environment Setup"]

T1 --> T2

T3["Verilator Installation"]

T2 --> T3

Set up development environment and verify installation

Overview

- This module covers the complete setup of your verification environment, including all required tool...

How to learn this module

Suggested learning path

- Clone the repo and read ``docs/MODULE0.md`` for toolchain expectations (GCC/Clang, Python 3.10+, Git,...)
- Confirm you have build essentials for Verilator (``sudo apt install build-essential`` on Debian/Ubuntu...)
- Decide install modes: Verilator (``submodule`/`system`/`source``), cocotb/pyuvvm (``pip`` or ``submodule``...)
- Plan for a project-local venv at ``.venv``; activate it (``source .venv/bin/activate``) before any Pyth...
- No RTL or simulation in this module — success means every tool probe passes before Module 1

Design architecture

1. Toolchain and repository layout

- Course root: ``docs/``,
``moduleN/``,
``scripts/moduleN.sh``,
``media/moduleN/`` for slides and video
- Python 3.10+ virtual environment (``.venv``) holds cocotb, pyuvvm, and course dependencies
- Verilator compiles RTL when modules include ``dut/``; cocotb bridges Python testbenches to the simula...
- Orchestrator
``scripts/module0.sh`` installs

```
Rtl Architecture
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
    subgraph host["Host environment"]
        PY[Python 3.10+ venv]
        GIT[Git / submodules]
        GCC[GCC or Clang + Make]
    end
end
```

2. Verification stack architecture

- Bottom layer: OS, compiler
(GCC/Clang), Make/Ninja, Git
- Simulator layer: Verilator
(default) produces fast cycle-accurate models from SystemVerilog/Verilog
- Testbench layer: cocotb
(coroutine-driven) and pyuvvm
(UVM 1.2 in Python) share the same venv
- Artifacts: logs under module runs, optional VCD/FST waveforms from Make targets in later modules

Verification Architecture

Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
DOC[docs/MODULE0.md] --> SCR[scripts/module0.sh]
SCR --> INS[install_verilator / cocotb / pyuvvm]
INS --> CHK[check smoke]
CHK --> M1[Module 1 labs]

3. Execution pipeline

- Install: ``module0.sh`` → dependency scripts → venv
population → version probes (no compile/sim/check...)
- Build: not applicable — no DUT or Makefile targets
in Module 0
- Sim: not applicable — cocotb/Verilator coupling is
validated only via import and ``--version`` probes
- Check: ``--verify-only`` aggregates pass/fail; all
three tools must report OK before Module 1 labs

4. What is not in this module

- No DUT or UVM hierarchy — focus is reproducible environment setup
- Later modules attach Python tests to RTL under ``moduleN/dut/``
- Self-check only validates tools; functional verification starts in Module 1
- Waveforms, scoreboards, and regression Makefiles appear from Module 1 onward

Install orchestrator (excerpt)

```
#!/bin/bash

# Module 0: Installation and Setup Orchestrator
# This script orchestrates the installation of all tools required for
Module 0
# Usage: ./module0.sh [OPTIONS]

set -euo pipefail

# Colors for output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
CYAN='\033[0;36m'
NC='\033[0m' # No Color
```

Key files to study

Open these in the repo

- ``scripts/module0.sh`` — orchestrator for Verilator, cocotb, and pyuvm install plus verification prob...
- ``scripts/install_verilator.sh`` — builds or links Verilator per ``--verilator-mode``
- ``scripts/install_cocotb.sh`` — installs cocotb into ``.venv`` or submodule tree
- ``scripts/install_pyuvm.sh`` — installs pyuvm and pins course Python dependencies
- ``docs/MODULE0.md`` — assessment checklist mapping environment readiness to course progress
- ``requirements.txt`` / ``pyproject.toml`` — dependency versions consumed by pip install paths

Verification & testing methods

1. Installation verification strategy

- Run `./scripts/module0.sh`` for full install or per-tool flags (`--verilator-mode``, `--cocotb-mode``, ...)
- `./scripts/module0.sh --verify-only`` confirms `verilator``, `cocotb-config``, and importable `pyuvm``
- Version probes (`verilator --version``, `python -c 'import cocotb'``) catch PATH and venv mistakes ea...

Testing Methods

Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
A[Run module0.sh] --> B{Tool present?}
B -->|no| C[Fix install layer]
B -->|yes| D[check pass]
D --> E[Ready for RTL testbenches]
C --> A

2. Smoke and regression mindset

- Treat install as the first regression: every tool must pass before Module 1 labs
- Re-run `--verify-only` after OS updates or submodule pulls
- Document failures in install logs; fix one layer at a time (compiler → Verilator → Python packages)

3. Step-by-step lab execution

- 1. From repo root: `./scripts/module0.sh`` (or selective `--skip-*`` flags if tools already installed)
- 2. Activate venv: `source .venv/bin/activate``
- 3. Verify: `./scripts/module0.sh --verify-only`` — expect green checks for Verilator, cocotb, pyuvm
- 4. Optional manual probes: `verilator --version``, `cocotb-config --version``, `python3 -c 'import py...`
- 5. Record pass in MODULE0 assessment checklist; proceed to Module 1 only when all probes succeed

4. Closure for Module 0

- Pass criteria: all install scripts succeed and
`--verify-only` reports required tools present
- Optional: run a minimal cocotb compile in Module 1
only after Module 0 passes
- Assessment checklist in `docs/MODULE0.md` maps
environment readiness to course progress

Syllabus topics

1. System Requirements and Prerequisites (1/3)

- Operating System Support
- Linux (Ubuntu/Debian, CentOS/RHEL, Fedora)
- macOS (Intel and Apple Silicon)
- Windows (WSL2 recommended)
- Hardware Requirements
- Minimum 4GB RAM (8GB+ recommended)

1. System Requirements and Prerequisites (2/3)

- 10GB free disk space
- Multi-core processor recommended
- Software Prerequisites
 - Python 3.8+ (3.10+ recommended)
 - Git
 - C/C++ compiler (GCC, Clang, or MSVC)

1. System Requirements and Prerequisites (3/3)

- Make or Ninja build system

2. Python Environment Setup (1/3)

- Python Installation
- Installing Python 3.10+ on Linux
- Installing Python 3.10+ on macOS
- Installing Python 3.10+ on Windows/WSL2
- Verifying Python installation
- Virtual Environment Management

2. Python Environment Setup (2/3)

- Using `venv` (Python built-in)
- Using `conda` (Anaconda/Miniconda)
- Using `uv` or `rye` (modern Python package managers)
- Best practices for virtual environments
- Package Management
- pip basics

2. Python Environment Setup (3/3)

- requirements.txt management
- Dependency resolution

3. Verilator Installation (1/6)

- What is Verilator?
- Open-source Verilog/SystemVerilog simulator
- Fast compilation and simulation
- Integration with cocotb
- Automated Installation (Recommended)
- Using the installation script:

3. Verilator Installation (2/6)

- The script automatically:
- Checks for existing installations
- Installs system dependencies (build tools, libraries)
- Sets up git submodule in `tools/verilator/`
- Builds and installs Verilator
- Verifies the installation

3. Verilator Installation (3/6)

- Manual Installation Methods
- Linux Installation
- Ubuntu/Debian: ``sudo apt-get install verilator``
- CentOS/RHEL: ``sudo yum install verilator`` or ``sudo dnf install verilator``
- Fedora: ``sudo dnf install verilator``
- Building from source (latest features)

3. Verilator Installation (4/6)

- macOS Installation
- Homebrew installation: ``brew install verilator``
- MacPorts installation (alternative)
- Building from source
- Windows/WSL2 Installation
- Installing in WSL2 Ubuntu: ``sudo apt-get install verilator``

3. Verilator Installation (5/6)

- Building from source in WSL2
- Verifying installation
- Uninstallation
- Verification Steps
- Check Verilator version: ``verilator --version``
- Run simple Verilator test

3. Verilator Installation (6/6)

- Verify compilation works

4. cocotb Installation and Verification (1/5)

- What is cocotb?
- Coroutine-based testbench framework
- Python testbenches for hardware
- Simulator abstraction layer
- Automated Installation (Recommended)
- Using the installation script:

4. cocotb Installation and Verification (2/5)

- The script automatically:
- Creates/uses virtual environment (default: ``.venv``)
- Sets up git submodule in ``tools/cocotb/`` (if using submodule mode)
- Installs dependencies
- Installs cocotb (via pip or from source)
- Verifies the installation

4. cocotb Installation and Verification (3/5)

- Manual Installation Methods
- pip installation (recommended):
- Development installation from source
- Version pinning for stability: ``pip install cocotb==2.0.0``
- Uninstallation
- Simulator Configuration

4. cocotb Installation and Verification (4/5)

- Verilator configuration
- Icarus Verilog configuration (alternative)
- ModelSim/QuartaSim configuration (if available)
- GHDL configuration (VHDL support)
- Environment Variables
- ``COCOTB_REDUCED_LOG_FMT``

4. cocotb Installation and Verification (5/5)

- `MODULE` and `TESTCASE`
- `SIM` variable for simulator selection
- Verification Steps
- Import cocotb successfully: ``python3 -c "import cocotb; print(cocotb.__version__)"``
- Run simple cocotb test
- Verify simulator integration

5. pyuvm Installation and Verification (1/5)

- What is pyuvm?
- Python implementation of UVM 1.2
- Works with cocotb
- Full UVM methodology support
- Automated Installation (Recommended)
- Using the installation script:

5. pyuvm Installation and Verification (2/5)

- The script automatically:
- Creates/uses virtual environment (default: ``.venv``)
- Sets up git submodule in ``tools/pyuvm/`` (if using submodule mode)
- Installs dependencies
- Installs pyuvm (via pip or from source)
- Verifies the installation

5. pyuvm Installation and Verification (3/5)

- Manual Installation Methods
- pip installation:
- Installation from source (development)
- Version selection and compatibility: ``pip install pyuvm==2.9.0``
- Uninstallation
- Dependencies

5. pyuvm Installation and Verification (4/5)

- Understanding pyuvm dependencies
- Resolving dependency conflicts
- Updating pyuvm
- Verification Steps
- Import pyuvm successfully: ``python3 -c "import pyuvm; print(pyuvm.__version__)"``
- Check pyuvm version

5. pyuvm Installation and Verification (5/5)

- Run simple pyuvm test
- Verify UVM classes available

6. IDE Setup and Configuration (1/4)

- Recommended IDEs
- VS Code with Python extension
- PyCharm (Community or Professional)
- Vim/Neovim with LSP
- Emacs with Python support
- VS Code Configuration

6. IDE Setup and Configuration (2/4)

- Python extension setup
- Pylance/Pyright configuration
- Debugging configuration
- Task runner setup for simulations
- Extension recommendations
- PyCharm Configuration

6. IDE Setup and Configuration (3/4)

- Python interpreter setup
- Virtual environment configuration
- Run configurations for tests
- Debugging setup
- Editor Configuration
- Python formatting (Black, Ruff)

6. IDE Setup and Configuration (4/4)

- Linting (pylint, flake8, ruff)
- Type checking (mypy, pyright)
- Code formatting on save

7. Project Structure Setup (1/4)

- Directory Structure
- Source code organization
- Test directory structure
- DUT (Design Under Test) organization
- Configuration files
- `tools/` directory for git submodules (Verilator, cocotb, pyuvvm)

7. Project Structure Setup (2/4)

- Git Submodules Management
- Tools are managed as git submodules in the `tools/` directory
- Initialize submodules:
`./scripts/init\_submodules.sh` or `git submodule update --init --recursive`
- Update submodules: `./scripts/update\_submodules.sh`
or `git submodule update --remote`
- Add new submodule: `./scripts/add\_submodule.sh`
`<repo\_url> <path>`
- Remove submodule: `./scripts/remove\_submodule.sh`
`<path>`

7. Project Structure Setup (3/4)

- Makefile/Configuration
- Simple Makefile for running tests
- pytest configuration
- cocotb Makefile setup
- Environment variable management
- Version Control

7. Project Structure Setup (4/4)

- Git initialization
- .gitignore for Python and simulation (include
 `.venv/`, `\_\_pycache\_\_`, build artifacts)
- Initial commit structure
- Git submodules in `.gitmodules` file

8. First "Hello World" Verification Test (1/3)

- Prerequisites
- Ensure all tools are installed:
 ``./scripts/module0.sh --verify-only``
- Activate virtual environment: ``source .venv/bin/activate`` (if using venv)
- Verify tools are accessible
- Simple DUT Creation
- Basic Verilog module (e.g., AND gate)

8. First "Hello World" Verification Test (2/3)

- Simple testbench structure
- cocotb Test
- Basic cocotb test structure
- Clock generation
- Signal driving and reading
- Running the test (requires Verilator and cocotb to be installed)

8. First "Hello World" Verification Test (3/3)

- pyuvm Test
- First UVM test class
- Basic UVM phases
- Running pyuvm test (requires cocotb and pyuvm to be installed)
- Understanding output

9. Troubleshooting Common Issues (1/4)

- Python Issues
 - Python version conflicts
 - Virtual environment activation problems
 - Package installation failures
- Verilator Issues
- Compilation errors

9. Troubleshooting Common Issues (2/4)

- Missing dependencies
- Version compatibility
- Path issues
- cocotb Issues
- Simulator not found
- Import errors

9. Troubleshooting Common Issues (3/4)

- Environment variable problems
- Module loading issues
- pyuvm Issues
- Import errors
- Version compatibility
- Dependency conflicts

9. Troubleshooting Common Issues (4/4)

- IDE Issues
- Python interpreter not found
- Import resolution problems
- Debugging not working

10. Verification Checklist (1/3)

- Python 3.10+ installed and working
- Virtual environment created and activated (or use ``./scripts/module0.sh`` which creates it automatic...
- Verilator installed and verified
- Using script: ``./scripts/install_verilator.sh --from-submodule``
- Verify: ``verilator --version``
- cocotb installed and verified

10. Verification Checklist (2/3)

- Using script: ``./scripts/install_cocotb.sh --pip --venv .venv``
- Verify: ``python3 -c "import cocotb; print(cocotb.__version__)"``
- pyuvm installed and verified
- Using script: ``./scripts/install_pyuvm.sh --pip --venv .venv``
- Verify: ``python3 -c "import pyuvm; print(pyuvm.__version__)"``
- All tools verified together: ``./scripts/module0.sh --verify-only``

10. Verification Checklist (3/3)

- IDE configured and working
- First test runs successfully
- Can create and run simple testbench
- Understand basic project structure
- Know how to get help when stuck

Command reference highlights

Full toolchain install

- ``./scripts/module0.sh`` — install Verilator, cocotb, and pyuvm with default modes into ``.venv``
- ``./scripts/module0.sh --verilator-mode system --cocotb-mode pip --pyuvm-mode pip`` — explicit mode S...
- ``./scripts/module0.sh --skip-verilator`` — refresh Python packages only when Verilator already prese...

Verify without reinstalling

- ``./scripts/module0.sh --verify-only`` — probe ``verilator``, ``cocotb-config``, and ``import pyuvvm`` witho...
- ``verilator --version`` — confirms simulator on PATH after submodule or system install
- ``python3 -c "import cocotb, pyuvvm; print('OK')"`` — quick venv sanity check outside the orchestrator

Manual probes (debug)

- ``which verilator && cocotb-config --version`` —
separate PATH issues from Python import failures
- ``source .venv/bin/activate`` — required before manual
Python probes if ``--no-venv`` was not used
- Re-run ``--verify-only`` after OS updates, submodule
pulls, or venv recreation

Hands-on examples

Module 0 orchestrator

```
$ ./scripts/module0.sh --help
```

Terminal: module0_check

```
[[0;36m===== [[0m
[[0;36mModule 0: Installation and Setup [[0m
[[0;36m===== [[0m

[[0;34m[2026-05-30 20:17:22] Starting Module 0 installation process... [[0m

[[0;34m[2026-05-30 20:17:22] Installation Plan: [[0m
  ✓ Verilator (mode: submodule)
  ✓ cocotb (mode: pip)
  ✓ pyuvvm (mode: pip)
  ✓ Virtual environment: /home/yongfu/proj/universal-verification-methodology/learn_uvm_pyuvvm/.v
Usage: ./scripts/module0.sh [OPTIONS]

Module 0: Installation and Setup Orchestrator
This script installs all tools required for pyuvvm verification environment.

OPTIONS:
  Installation Control:
    --skip-verilator  Skip Verilator installation
    --skip-cocotb     Skip cocotb installation
    --skip-pyuvvm     Skip pyuvvm installation

  Installation Modes:
    --verilator-mode MODE  Verilator installation mode: submodule (default), system, source
    --cocotb-mode MODE     cocotb installation mode: pip (default), submodule
    --pyuvvm-mode MODE     pyuvvm installation mode: pip (default), submodule

  Environment:
    --venv DIR            Virtual environment directory (default: .venv)
    --no-venv             Install in system Python (not recommended)

  Other:
    --skip-checks        Skip pre-installation checks
    --verify-only        Only verify installations (don't install)
    --help, -h          Show this help message
```

Exercise scaffold

```
./scripts/module0.sh --scaffold
```

Practice & assessment

What you should know (1/2)

- Install and configure all required tools
- Set up a Python virtual environment
- Install and verify Verilator
- Install and verify cocotb
- Install and verify pyuvvm
- Configure your IDE for verification work

What you should know (2/2)

- Create a basic project structure
- Run a simple verification test
- Troubleshoot common installation issues

Exercises

- Installation Verification
- Environment Setup
- First Test
- IDE Configuration
- Project Structure

Assessment checklist

- Can install all required tools independently
- Can set up Python virtual environment
- Can verify Verilator installation
- Can verify cocotb installation
- Can verify pyuvvm installation
- Can configure IDE for verification work

Summary & next steps

- Set up development environment and verify installation
- Verify tools: verilator, cocotb, pyuvm (see MODULE0.md)
- Review module0/EXAMPLES.md and run each lab
- Next: Next module in course